

tf.compat.v1.nn.pool

Stay organized with collections

Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/pool

Performs an N-D pooling operation.

Function Signatures

```
tf.compat.v1.nn.pool( input, window_shape, pooling_type,
padding, dilation_rate=None, strides=None, name=None,
data_format=None, dilations=None )
```

Code Examples

```
tf.compat.v1.nn.pool(
input,
window_shape,
pooling_type,
padding,
dilation_rate=None,
strides=None,
name=None,
data_format=None,
dilations=None
)
```

```
tf.compat.v1.nn.pool(
input,
window_shape,
pooling_type,
padding,
dilation_rate=None,
strides=None,
name=None,
data_format=None,
dilations=None
)
```

```
output[b, x[0], ..., x[N-1], c] =
REDUCE_{z[0], ..., z[N-1]}
input[b,
x[0] * strides[0] - pad_before[0] + dilation_rate[0]*z[0],
...
x[N-1]*strides[N-1] - pad_before[N-1] + dilation_rate[N-1]*z[N-1],
c],
```

```
output[b, x[0], ..., x[N-1], c] =
REDUCE_{z[0], ..., z[N-1]}
input[b,
x[0] * strides[0] - pad_before[0] + dilation_rate[0]*z[0],
...
x[N-1]*strides[N-1] - pad_before[N-1] + dilation_rate[N-1]*z[N-1],
c],
```

```
pool(input, data_format, **kwargs) =
tf.transpose(pool(tf.transpose(input, [0] + range(2,N+2) + [1]),
**kwargs),
[0, N+1] + range(1, N+1))
```

```
pool(input, data_format, **kwargs) =  
tf.transpose(pool(tf.transpose(input, [0] + range(2,N+2) + [1]),  
**kwargs),  
[0, N+1] + range(1, N+1))
```

Documentation

Performs an N-D pooling operation.

In the case that `data_format` does not start with "NC", computes for

$0 \leq b < \text{batch_size}$,

$0 \leq x[i] < \text{output_spatial_shape}[i]$,

$0 \leq c < \text{num_channels}$:

where the reduction function REDUCE depends on the value of `pooling_type`, and `pad_before` is defined based on the value of padding as described in the "returns" section of `tf.nn.convolution` for details.

The reduction never includes out-of-bounds positions.

In the case that `data_format` starts with "NC", the input and output are simply transposed as follows:

Returns

if `data_format` is `None` or does not start with "NC", or

`[batch_size, num_channels] + output_spatial_shape`

if `data_format` starts with "NC",

where `output_spatial_shape` depends on the value of padding:

If padding = "SAME":

output_spatial_shape[i] = ceil(input_spatial_shape[i] / strides[i])

If padding = "VALID":

output_spatial_shape[i] =

ceil((input_spatial_shape[i] - (window_shape[i] - 1) * dilation_rate[i])

/ strides[i]).

Raises